

Day 8 – Daily Submission Report

Build a RAG Application

Date: 28 September 2026

Topic: Build a Company Knowledge Assistant using Retrieval-Augmented Generation (RAG)

What I Learned:

I learned how Retrieval-Augmented Generation (RAG) applications combine document retrieval with a Large Language Model (LLM) to answer questions using information from a specific knowledge base.

I learned about the main components of a RAG pipeline, including:

- Document ingestion
- Text chunking
- Embeddings
- Vector databases
- Similarity search
- Context retrieval
- LLM generation
- Source and reference information

I also learned that a RAG application should not depend only on the LLM. Relevant information should first be retrieved from the knowledge base and then provided to the LLM as context.

In addition, I learned the importance of error handling when working with external LLM APIs.

What I Implemented:

I implemented a **Company Knowledge Assistant** as a RAG application using sample company and project documents.

The application follows this process:

Documents → Chunking → Embeddings → ChromaDB → Similarity Search → Context Retrieval → Gemini → Answer

The implementation includes:

- Reading company/project documents from the data folder
- Splitting documents into smaller chunks
- Generating embeddings using the all-MiniLM-L6-v2 Sentence Transformer model
- Storing document embeddings and metadata in ChromaDB
- Converting user questions into embeddings
- Performing similarity search to retrieve relevant document chunks
- Passing the retrieved information to Gemini as context
- Generating answers based only on the retrieved context
- Providing source filename information with the retrieved content
- Handling empty user questions
- Allowing the user to exit the application using the exit command

I also added Gemini API error handling. If the primary Gemini model returns a `ServerError`, the application attempts to use a fallback Gemini Flash-Lite model.

What Worked:

- The document ingestion process worked successfully.
- The documents were divided into smaller chunks and converted into embeddings.
- The generated embeddings were successfully stored in ChromaDB together with metadata such as the source filename and chunk number.
- The application was able to perform semantic similarity searches when a user entered a question.
- Relevant document chunks were retrieved and passed to Gemini as context for answer generation.
- The application also includes instructions to avoid generating information that is not available in the retrieved documents.

What Failed:

- During development, the Gemini API could return a `ServerError` when the primary model was temporarily unavailable.
- This created a problem because the application depended on the primary Gemini model to generate the final response.
- Another limitation identified during testing is that semantic retrieval may sometimes return information that is only partially related to a question.

How I Solved It:

I imported the `ServerError` exception from the Google GenAI package and added error handling around the Gemini API request.

When the primary Gemini model returns a `ServerError`, the application attempts to use a fallback Gemini Flash-Lite model.

I also added another error-handling layer in the main application so that the program does not terminate immediately if the Gemini service is unavailable.

For the RAG prompt, I added instructions telling the model to:

- Use only the retrieved context
- Avoid inventing information
- Clearly state when the requested information is not available
- Include the source filename used for the answer

What I Would Improve:

I would improve the document chunking strategy for larger and more complex documents.

I would experiment with different chunk sizes and overlap values to determine which settings provide better retrieval results.

I would also test different Top-K retrieval values to determine how many document chunks should be provided to the LLM.

Another improvement would be to provide a clearer source/reference section in the final response.

For a production-level application, I would also add a web-based user interface instead of using only the command-line interface.

Practical Testing:

The RAG assistant will be tested using at least ten questions covering:

1. Questions with information that exists in the knowledge base
2. Questions requiring information from different documents
3. Questions where the retrieved information may be incomplete
4. Questions asking for information that does not exist in the knowledge base

The test results will be recorded to identify whether retrieval was correct, incomplete, or incorrect.

```
(.venv) PS E:\VisionRoot-AI engineer Internship\GitHub\AI-Engineer-Training\AI-Engineer-Training\Week-02\Day-08\practical-task> python ingest.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 2428.

Processing: company_overview.txt
Created 2 chunks

Processing: development.txt
Created 2 chunks

Processing: policies.txt
Created 2 chunks

Processing: products.txt
Created 2 chunks

=====
DOCUMENT INGESTION COMPLETED
=====
```

```
y-08\practical-task> python rag_app.py
```

Ask questions about the available documents.

Type 'exit' to close the application.

Question: what is product A?

Searching knowledge base...

Generating answer...

Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.

=====

ANSWER

=====

****Answer:****

Product A is a customer management web application that allows users to create customer records, update customer information, search customer records, and view customer details.

****Sources:****

- products.txt

Question: What can users do with Product A?

Searching knowledge base...

Generating answer...

Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.

=====

ANSWER

=====

1. Answer:

With Product A, users can create customer records, update customer information, search customer records, and view customer details.

2. Sources:

products.txt

Question: []

Type 'exit' to close the application.

Question: What is Product B?

Searching knowledge base...

Generating answer...

Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.

=====

ANSWER

=====

****Answer:****

Product B is an AI knowledge assistant that allows users to ask questions about approved documents. The system retrieves relevant document sections and provides them as context to a language model, and it should provide source information when answering questions.

****Sources:****

products.txt

Question: []

Question: How does the AI knowledge assistant work?

Searching knowledge base...

Generating answer...

Direct use of automatic function calling (AFC) in `Models.generate_content` is not recommended. Instead, we recommend to use AFC in `Chat.send_message`. Similarly, direct use of AFC in `Models.generate_content_stream` is not recommended. Instead, we recommend to use AFC in `Chat.send_message_stream`.

Gemini 3.8 Flash is temporarily unavailable; trying Gemini Flash Lite...

=====
ANSWER
=====

1. Answer:

The AI knowledge assistant works by allowing users to ask questions about approved documents. The system retrieves relevant document sections and provides them as context to a language model. Additionally, the knowledge assistant provides source information when answering questions.

2. Sources:

- products.txt

Question:

Question: What is Git used for?

Searching knowledge base...

Generating answer...

=====
ANSWER
=====

1. Answer:

Git is used for source code version control.

2. Sources:

- company_overview.txt

- development.txt

Question:

Question: What should developers do when an unexpected error occurs?

Searching knowledge base...

Generating answer...

Gemini 3.8 Flash is temporarily unavailable; trying Gemini Flash Lite...

=====
ANSWER
=====

1. Answer: When an application produces an unexpected result, developers should reproduce the problem, inspect the error, identify the cause, apply a fix, and test the solution again.

2. Sources: development.txt

Question: What types of documents should be added to the knowledge base?

Searching knowledge base...

Generating answer...

=====
ANSWER
=====

1. Answer:

Project documents that have been reviewed should be added to the knowledge base, and only approved documents should be used as knowledge sources.

2. Sources:

policies.txt

Question: What is the company's annual revenue?

Searching knowledge base...

Generating answer...

=====

ANSWER

=====

1. Answer:

I could not find this information in the available company documents.

2. Sources:

None

Question: Who is the CEO of the company?

Searching knowledge base...

Generating answer...

Gemini 3.8 Flash is temporarily unavailable; trying Gemini Flash Lite...

=====

ANSWER

=====

1. Answer:

I could not find this information in the available company documents.

2. Sources:

None